

DEEP LEARNING FRAMEWORK FOR SOLVING PARTIAL DIFFERENTIAL EQUATIONS

A Project Report Submitted
for the Course

MA498 Project I

by

Rakesh Kumar

Roll No. 220123051



to the

**DEPARTMENT OF MATHEMATICS
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI
GUWAHATI - 781039, INDIA**

November 2025

CERTIFICATE

This is to certify that the work contained in this project report entitled “Deep learning framework for solving Partial Differential Equations” submitted by Rakesh Kumar (Roll No.: 220123051) to the Department of Mathematics, Indian Institute of Technology Guwahati towards partial requirement of Bachelor of Technology in Mathematics and Computing has been carried out by him/her under my supervision.

It is also certified that this report is a survey work based on the references in the bibliography.

Turnitin Similarity: 24%

Guwahati - 781 039
November 2025

Dr. Satyajit Pramanik
Project Supervisor

ABSTRACT

This project develops a **deep learning framework** based on **Physics-Informed Neural Networks (PINNs)** to solve **Partial Differential Equations (PDEs)**. The method embeds physical laws into the neural network loss function, enabling accurate, mesh-free solutions without labeled data. A detailed study of machine learning concepts and deep neural architectures was conducted, followed by the implementation of a PINN for the **one-dimensional wave equation**. The model achieved excellent agreement with the analytical solution while satisfying boundary and initial conditions, demonstrating the potential of PINNs for solving complex physical problems.

Contents

List of Figures	vi
1 Machine Learning Methods	1
1.1 Introduction	1
1.2 Supervised Learning	2
1.2.1 Common Supervised Learning Algorithms	3
1.3 Unsupervised Learning	4
1.4 Reinforcement Learning	5
1.5 Conclusion	6
2 Introduction to Deep Learning	7
2.1 Introduction	7
2.2 Basic Structure of a Neural Network	7
2.3 Forward Propagation	9
2.4 Activation Functions	10
2.5 Backward Propagation	11
2.6 Loss Functions	12
2.7 Optimization Algorithms	12
2.8 Conclusion	13

3	Physics-Informed Neural Networks (PINNs)	14
3.1	Introduction	14
3.2	Why Choose PINNs for Solving PDEs	14
3.2.1	Limitations of Traditional PDE Solvers	15
3.2.2	Advantages of PINNs	15
3.2.3	Why PINNs for PDEs	16
3.3	Basic Idea of PINNs	16
3.4	PINN Loss Function	17
3.5	PINN Training Workflow	17
3.6	Architecture of PINNs	18
3.7	Advantages in Practice	18
3.8	Conclusion	19
4	Implementation and Results	20
4.1	Introduction	20
4.2	Mathematical Formulation of the Wave Equation	20
4.2.1	Boundary and Initial Conditions	21
4.3	PINN Formulation for the Wave Equation	21
4.4	Loss Function	22
4.5	Network Architecture and Training	22
4.6	Results	23
4.6.1	Error Analysis	24
4.6.2	Observations	25
4.7	Conclusion	26
5	Conclusion and Future Direction	27
	Bibliography	28

List of Figures

1.1	Conceptual difference between classical programming and machine learning. In classical programming, rules and input values produce outputs, whereas in machine learning, input and output data are used to infer the rules [3].	2
2.1	Artificial Neural Network (ANN) architecture showing input, hidden, and output layers. Each neuron in one layer is connected to all neurons in the next layer [3].	9
3.1	Schematic of a PINN: the network takes $(\mathbf{x}, t)$ as inputs and outputs $u_\theta(\mathbf{x}, t)$. PDE residuals computed via automatic differentiation contribute to the loss [11].	18
4.1	Training loss history over 6,000 epochs showing total, PDE, boundary, and initial losses. The Adam optimizer achieves stable convergence[7].	23
4.2	Comparison between (a) PINN-predicted solution $u_\theta(x, t)$, (b) analytical solution $u(x, t)$, and (c) absolute error $ u - u_\theta $. The PINN accurately reproduces the full wave pattern[7].	24

4.3 Comparison of $u(x, t)$ at different time instances ($t = 0, 0.25, 0.5, 0.75$, and 1.0). The PINN-predicted and analytical curves nearly overlap, demonstrating precise temporal predictions[7]. 25

Chapter 1

Machine Learning Methods

1.1 Introduction

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that enables computers to learn from data without being explicitly programmed. It focuses on developing models that can recognize patterns, make predictions, and adapt from experience [1, 3]. ML algorithms are generally classified into three main categories: *supervised learning*, *unsupervised learning*, and *reinforcement learning* [9].

In scientific computing and the study of partial differential equations (PDEs), ML and deep learning are increasingly used to approximate complex mappings, discover hidden relationships, and accelerate numerical solvers [11, 5]. The following figure illustrates the conceptual difference between classical programming and machine learning.

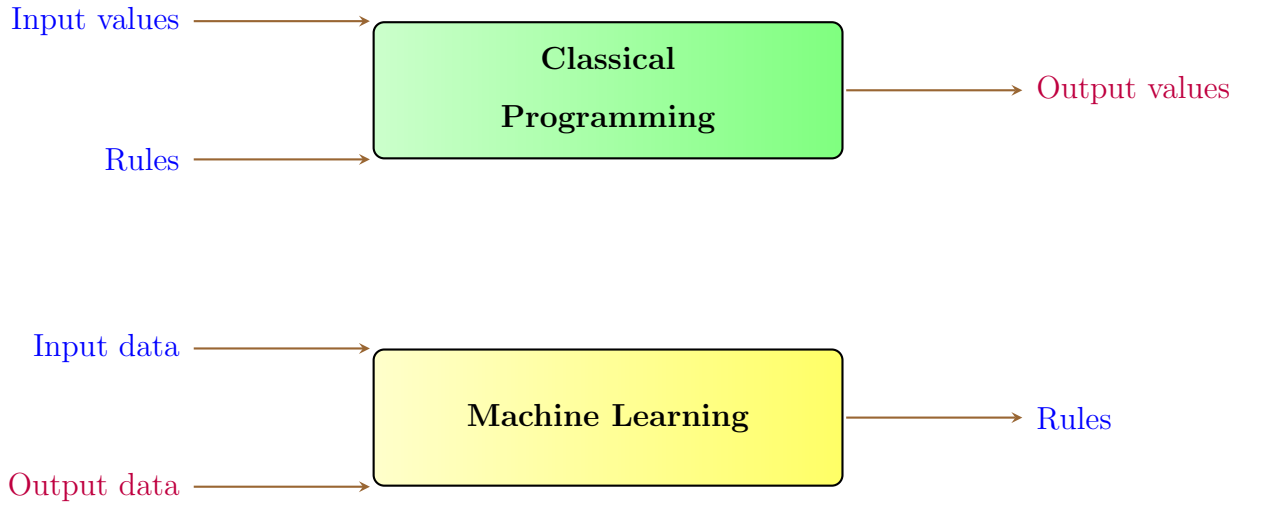


Figure 1.1: Conceptual difference between classical programming and machine learning. In classical programming, rules and input values produce outputs, whereas in machine learning, input and output data are used to infer the rules [3].

1.2 Supervised Learning

Supervised learning involves training a model on labeled data, where each input x_i has a corresponding output y_i . The goal is to learn a function $f_\theta : X \rightarrow Y$ that maps inputs to outputs by minimizing a loss function that measures prediction error:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - f_\theta(x_i))^2. \quad (1.1)$$

Once trained, the model can make predictions on unseen data.

1.2.1 Common Supervised Learning Algorithms

Linear and Logistic Regression

Linear Regression models a linear relationship between features $\mathbf{x} = [x_1, x_2, \dots, x_n]$ and the target y :

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b. \quad (1.2)$$

Parameters $\mathbf{w}$ and b are optimized by minimizing the Mean Squared Error (MSE):

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2. \quad (1.3)$$

Logistic Regression, though called regression, is used for classification. It predicts probabilities using the sigmoid function:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}. \quad (1.4)$$

It is efficient for linearly separable classification tasks [1].

Support Vector Machines (SVM)

Support Vector Machines (SVM) aim to find the optimal separating hyperplane between different data classes [9]. For linearly separable data, the decision boundary is defined as:

$$\mathbf{w}^\top \mathbf{x} + b = 0, \quad (1.5)$$

where $\mathbf{w}$ represents the normal vector to the hyperplane and b is the bias term. SVM maximizes the margin—the distance between the hyperplane and the closest data points—ensuring better generalization. For nonlinear problems, kernel functions such as the Radial Basis Function (RBF) can map

input data to higher-dimensional spaces.

Neural Networks

Neural Networks consist of layers of interconnected neurons that learn complex nonlinear mappings between inputs and outputs [3]. A simple feedforward neural network with one hidden layer can be expressed as:

$$\hat{y} = f(\mathbf{x}) = \sigma_2(\mathbf{W}_2 \sigma_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2), \quad (1.6)$$

where σ_1 and σ_2 are activation functions (e.g., ReLU, sigmoid), and $\mathbf{W}_1, \mathbf{W}_2$ are weight matrices. Training is performed using the *backpropagation algorithm*, which minimizes the loss function through gradient-based optimization. Neural networks form the foundation of modern deep learning and serve as powerful function approximators for solving PDEs [11].

1.3 Unsupervised Learning

In unsupervised learning, the model is trained on *unlabeled data* with the goal of identifying patterns or structures within the dataset [4]. Unlike supervised learning, there are no target outputs; instead, the algorithm seeks to understand underlying distributions or relationships.

Key unsupervised learning methods include:

- **Clustering (e.g., K-Means):** Used to partition data into distinct groups based on similarity. K-Means iteratively assigns data points to the nearest cluster centroid and updates centroids based on the mean of assigned points, minimizing intra-cluster variance.
- **Principal Component Analysis (PCA):** A dimensionality reduc-

tion technique that identifies principal components capturing the maximum variance in data. It projects data $\mathbf{X}$ onto a lower-dimensional subspace using eigenvectors of the covariance matrix $\mathbf{X}^\top \mathbf{X}$.

- **Anomaly Detection:** A technique used to identify rare or unusual data points that deviate significantly from normal patterns. It can be implemented via statistical models, clustering, or autoencoders.

1.4 Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning where an agent learns to make decisions by interacting with an environment [14]. Instead of learning from a fixed dataset, the agent explores and receives feedback in the form of rewards or penalties based on its actions. Over time, the agent improves its decision-making strategy, known as a *policy*, to maximize long-term rewards.

Common RL techniques include:

- **Q-Learning:** A value-based method where the agent learns the best action for each situation by estimating expected future rewards.
- **Deep Q-Networks (DQN):** An advanced approach that combines Q-learning with deep neural networks, enabling agents to handle high-dimensional environments such as games, robotics, and control systems.

Reinforcement learning has wide applications in robotics, control theory, autonomous systems, and scientific computing, particularly in optimizing PDE-based simulations and adaptive algorithms [13].

1.5 Conclusion

This chapter introduced the three main paradigms of machine learning—supervised, unsupervised, and reinforcement learning. It discussed key algorithms such as Linear and Logistic Regression, Support Vector Machines, and Neural Networks, which together form the foundation of deep learning. The concepts presented here serve as the basis for the following chapter, where deep learning frameworks are explored for efficiently solving partial differential equations.

Chapter 2

Introduction to Deep Learning

2.1 Introduction

Deep Learning (DL) is a subset of machine learning that utilizes multi-layered neural networks to automatically learn hierarchical representations of data [3]. Unlike traditional machine learning algorithms that depend heavily on handcrafted features, deep learning models can automatically discover complex patterns and abstractions from raw data [8]. This property makes them particularly effective for modeling high-dimensional systems and nonlinear mappings such as those arising in partial differential equations (PDEs).

2.2 Basic Structure of a Neural Network

A neural network is a computational model inspired by the structure and functioning of biological neurons [1]. It consists of multiple layers of interconnected nodes (neurons) that transform input data through a series of weighted linear combinations followed by nonlinear activation functions.

A typical feedforward neural network comprises three types of layers:

- **Input Layer:** Receives the input data and feeds it into the network.
- **Hidden Layers:** Intermediate layers where weighted transformations and nonlinear activations extract complex features from the data.
- **Output Layer:** Produces the final predictions or classifications.

Mathematically, a single layer with input $\mathbf{x} \in \mathbb{R}^n$, weights $\mathbf{W}$, and bias $\mathbf{b}$ performs:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad (2.1)$$

and applies an activation function $\sigma(\cdot)$ to produce the layer output:

$$\mathbf{a} = \sigma(\mathbf{z}). \quad (2.2)$$

The overall mapping of a neural network with L layers is:

$$\hat{y} = f(\mathbf{x}) = \sigma_L(\mathbf{W}_L \sigma_{L-1}(\cdots \sigma_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \cdots) + \mathbf{b}_L). \quad (2.3)$$

The structure of a typical feedforward Artificial Neural Network (ANN) is illustrated in Figure 2.1 [3, 1].

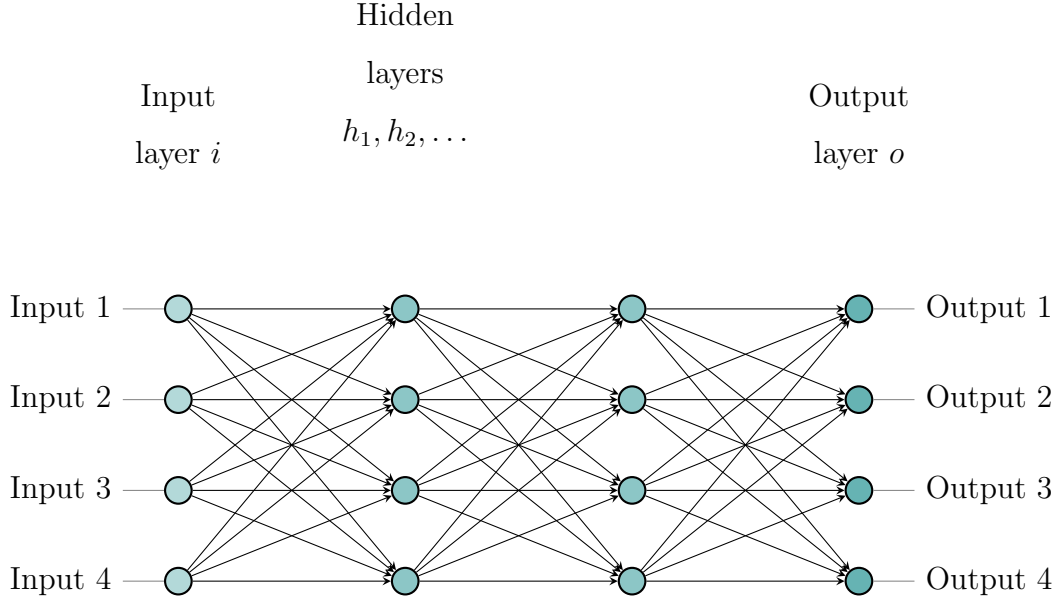


Figure 2.1: Artificial Neural Network (ANN) architecture showing input, hidden, and output layers. Each neuron in one layer is connected to all neurons in the next layer [3].

2.3 Forward Propagation

Forward propagation is the process of passing input data through the network to compute predictions [1]. Each neuron performs a weighted sum of its inputs, adds a bias term, and applies a nonlinear activation function. This sequential computation transforms the raw input into higher-level feature representations across layers.

Mathematically, for each layer l :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}),$$

where $\mathbf{a}^{(0)} = \mathbf{x}$ represents the input features. Forward propagation continues through all layers until the output $\hat{y}$ is produced.

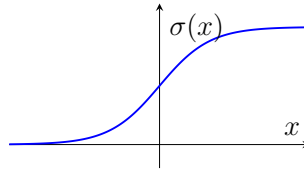
2.4 Activation Functions

Activation functions introduce nonlinearity into the network, allowing neural networks to model complex, nonlinear relationships between input and output variables [3, 1]. Without them, a multilayer network would behave as a single linear model regardless of its depth. Common activation functions include the following:

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

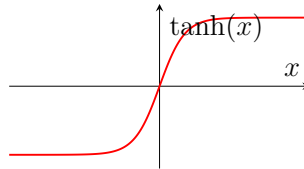
Smoothly squashes real values into the range $(0, 1)$ and is often used for binary classification. However, it can cause vanishing gradients in deep networks.



Hyperbolic Tangent (Tanh):

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

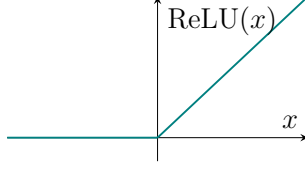
Maps input to $(-1, 1)$, is zero-centered, and typically converges faster than sigmoid.



Rectified Linear Unit (ReLU):

$$\text{ReLU}(x) = \max(0, x)$$

Efficient and widely used in deep networks due to its simplicity and ability to alleviate vanishing gradients [10].



Smooth activation functions like *tanh* or sinusoidal functions are often used in Physics-Informed Neural Networks (PINNs) for PDE problems, since they possess continuous derivatives required for computing higher-order terms [11, 5].

2.5 Backward Propagation

Backpropagation computes gradients of the loss with respect to parameters using the chain rule and updates them to reduce error [12]. The gradient of the loss with respect to a weight $w_{ij}^{(l)}$ in layer l is

$$\frac{\partial \text{Loss}}{\partial w_{ij}^{(l)}} = \frac{\partial \text{Loss}}{\partial a_i^{(l)}} \cdot \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \quad (2.4)$$

where

- $\frac{\partial \text{Loss}}{\partial a_i^{(l)}}$ is the loss sensitivity to activation,
- $\frac{\partial a_i^{(l)}}{\partial z_i^{(l)}}$ is the derivative of the activation function,
- $\frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}} = a_j^{(l-1)}$ arises from $z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$.

Backprop efficiently computes these gradients layer-by-layer; parameters are updated by optimization algorithms such as SGD or Adam [3, 1].

2.6 Loss Functions

The loss function quantifies how well the model’s predictions match the true outputs. It acts as a feedback signal guiding the learning process [1].

Common loss functions include:

- **Mean Squared Error (MSE):**

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2,$$

suitable for regression tasks.

- **Cross-Entropy Loss:**

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)],$$

commonly used for classification.

2.7 Optimization Algorithms

Optimization algorithms minimize the loss function by iteratively updating model parameters [3]. The basic update rule in **Gradient Descent** is:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}.$$

Popular variants include:

- **Stochastic Gradient Descent (SGD):** Uses small batches of data for faster but noisier updates.
- **Momentum:** Accelerates convergence by incorporating a fraction of the previous update.
- **Adam Optimizer:** Combines momentum and adaptive learning rates for efficient training [6].

Choosing an appropriate optimizer and learning rate is crucial for stable and efficient training, particularly when dealing with PDE-related networks where loss landscapes can be highly nonlinear [5].

2.8 Conclusion

This chapter presented the foundational elements of deep learning, including the structure of neural networks, forward and backward propagation, activation functions, loss functions, and optimization techniques. Each of these components contributes to how neural networks learn complex non-linear mappings from data. In the next chapter, we extend these ideas to *Physics-Informed Neural Networks (PINNs)*, which integrate physical laws and PDE constraints directly into the training process [11, 5].

Chapter 3

Physics-Informed Neural Networks (PINNs)

3.1 Introduction

Physics-Informed Neural Networks (PINNs) are deep learning models that incorporate the governing physical laws of a system—expressed as partial differential equations (PDEs)—directly into the training objective [11, 5]. Unlike purely data-driven models, PINNs jointly enforce data fidelity and the PDE residual, making them particularly valuable when data are scarce but the physics are well understood. They exploit automatic differentiation to compute derivatives required by the PDE operator, enabling smooth and differentiable approximations over continuous domains.

3.2 Why Choose PINNs for Solving PDEs

Classical discretization-based solvers (finite difference, finite volume, and finite element methods) have been extraordinarily successful. However, mod-

ern scientific problems often expose several limitations of these approaches [5, 2].

3.2.1 Limitations of Traditional PDE Solvers

- **Curse of dimensionality:** Computational cost grows exponentially with dimensionality, making high-dimensional PDEs intractable on dense meshes.
- **Meshing challenges:** Generating high-quality meshes for complex or moving geometries is difficult and time-consuming.
- **Noisy/sparse data:** Classical solvers assume precise boundary/initial data; real experiments may provide only scattered, noisy measurements.
- **Discrete, non-differentiable outputs:** Many workflows require differentiable surrogates (e.g., inverse problems, gradient-based design); grid outputs can be limiting.

3.2.2 Advantages of PINNs

PINNs mitigate these issues by unifying data and physics within a single optimization problem [11, 5].

- **Mesh-free:** Enforce physics at collocation points without spatial meshing.
- **Unified learning:** Combine observational data and PDE constraints in one loss.

- **Automatic differentiation:** Exact (up to machine precision) derivatives via backpropagation—no finite-difference truncation error.
- **Inverse problems:** Infer unknown parameters or sources together with the state.
- **Fast inference:** Trained networks act as differentiable surrogates for rapid queries.

3.2.3 Why PINNs for PDEs

For PDEs, the core idea is to learn a smooth function $u_\theta(\mathbf{x}, t)$ that *globally* satisfies the physics by driving the PDE residual toward zero while honoring data/BC/IC constraints:

$$\mathcal{N}[u_\theta(\mathbf{x}, t)] \approx 0 \quad \text{in } \Omega \times [0, T].$$

This continuous, differentiable representation makes PINNs well-suited to forward and inverse PDE problems alike [5, 2].

3.3 Basic Idea of PINNs

Consider a PDE on domain $\Omega \subset \mathbb{R}^d$ and time interval $[0, T]$:

$$\mathcal{N}[u(\mathbf{x}, t)] = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, T], \quad (3.1)$$

with initial and boundary conditions

$$u(\mathbf{x}, 0) = u_0(\mathbf{x}) \quad (\text{IC}), \quad (3.2)$$

$$\mathcal{B}[u(\mathbf{x}, t)] = g(\mathbf{x}, t) \quad (\text{BC}). \quad (3.3)$$

A neural network $u_\theta(\mathbf{x}, t)$ is used to approximate $u(\mathbf{x}, t)$. Using automatic differentiation, we compute all derivatives required by the operator $\mathcal{N}$ and form residuals at collocation points.

3.4 PINN Loss Function

The usual PINN objective couples data consistency and physics:

$$\mathcal{L}_{\text{PINN}} = \underbrace{\frac{1}{N_d} \sum_{i=1}^{N_d} |u_\theta(\mathbf{x}_i, t_i) - u_i|^2}_{\mathcal{L}_{\text{data}}} + \lambda_{\text{PDE}} \underbrace{\frac{1}{N_f} \sum_{j=1}^{N_f} |\mathcal{N}[u_\theta(\mathbf{x}_j, t_j)]|^2}_{\mathcal{L}_{\text{PDE}}}, \quad (3.4)$$

optionally augmented with boundary/initial terms (Dirichlet, Neumann, Robin) as data losses. The weight λ_{PDE} balances data fit and physics enforcement [11]. Training typically uses Adam and/or L-BFGS for robust convergence [6, 5].

3.5 PINN Training Workflow

1. **Specify physics:** PDE operator $\mathcal{N}$, domain Ω , time interval, and BC/IC types.
2. **Choose network:** Depth, width, activation (often `tanh`/sine for smooth derivatives).
3. **Sample points:** Data points $(\mathbf{x}_i, t_i)$; collocation points $(\mathbf{x}_j, t_j)$ for residuals (Latin Hypercube / uniform / adaptive sampling).
4. **Compute residuals:** Use automatic differentiation to evaluate $\mathcal{N}[u_\theta]$ and boundary operators.

5. **Optimize:** Minimize $\mathcal{L}_{\text{PINN}}$ with gradient-based optimizers.
6. **Validate:** Compare against analytical or high-fidelity numerical solutions.

3.6 Architecture of PINNs

A PINN looks like a standard fully connected network; the key addition is the physics residual pathway entering the loss.

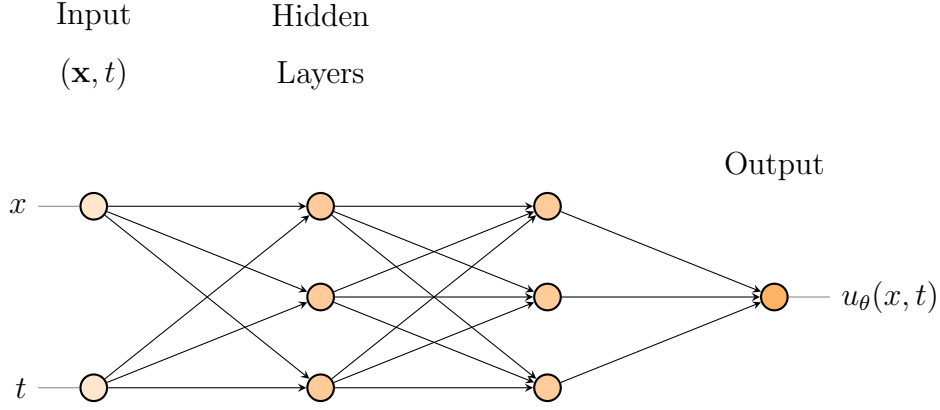


Figure 3.1: Schematic of a PINN: the network takes $(\mathbf{x}, t)$ as inputs and outputs $u_\theta(\mathbf{x}, t)$. PDE residuals computed via automatic differentiation contribute to the loss [11].

3.7 Advantages in Practice

- **Forward problems:** Learn $u(\mathbf{x}, t)$ consistent with physics and data.
- **Inverse problems:** Learn unknown parameters in $\mathcal{N}$ (e.g., diffusivity) alongside $u(\mathbf{x}, t)$.

- **Irregular data:** Handle scattered, partial, or noisy measurements in a principled way.
- **Differentiable surrogate:** Once trained, enables fast queries and sensitivity analysis.

3.8 Conclusion

PINNs bridge data-driven learning and physics-based modeling by embedding PDE residuals directly into the training objective. Their mesh-free, differentiable formulation makes them attractive for forward and inverse PDE tasks, especially when data are limited but governing laws are known [11, 5, 2]. Subsequent sections can focus on concrete case studies (e.g., Poisson, Burgers, or Navier–Stokes) and implementation details such as collocation strategies and loss balancing.

Chapter 4

Implementation and Results

4.1 Introduction

In this chapter, we implement and analyze the Physics-Informed Neural Network (PINN) framework for solving a representative hyperbolic partial differential equation — the one-dimensional wave equation. The wave equation describes oscillatory motion such as sound or vibration propagation and provides a fundamental benchmark to evaluate PINN performance [11, 5].

4.2 Mathematical Formulation of the Wave Equation

The one-dimensional wave equation is given by:

$$u_{tt} = c^2 u_{xx}, \quad x \in (0, 1), \quad t \in (0, 1), \quad (4.1)$$

where $u(x, t)$ represents the displacement field and c is the wave propagation speed (here, $c = 1$).

4.2.1 Boundary and Initial Conditions

The following conditions are imposed:

$$u(0, t) = u(1, t) = 0, \quad (\text{Dirichlet boundary conditions}), \quad (4.2)$$

$$u(x, 0) = \sin(\pi x), \quad (\text{Initial displacement}), \quad (4.3)$$

$$u_t(x, 0) = 0, \quad (\text{Initial velocity}). \quad (4.4)$$

The analytical solution corresponding to these conditions is:

$$u(x, t) = \sin(\pi x) \cos(\pi t),$$

which serves as the reference to assess the PINN performance.

4.3 PINN Formulation for the Wave Equation

The neural network $u_\theta(x, t)$ parameterized by θ approximates the wave field. The PDE residual is computed via automatic differentiation as:

$$f_\theta(x, t) = u_{tt} - c^2 u_{xx}. \quad (4.5)$$

The network minimizes the residual while satisfying the boundary and initial conditions through a composite loss function.

4.4 Loss Function

The total loss is formulated as:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}} + \mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{BC}}, \quad (4.6)$$

where

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f} \sum_{i=1}^{N_f} |f_{\theta}(x_i, t_i)|^2, \quad (4.7)$$

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{\text{IC}}} \sum_{i=1}^{N_{\text{IC}}} (|u_{\theta}(x_i, 0) - \sin(\pi x_i)|^2 + |u_t(x_i, 0)|^2), \quad (4.8)$$

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_{\text{BC}}} \sum_{i=1}^{N_{\text{BC}}} (|u_{\theta}(0, t_i)|^2 + |u_{\theta}(1, t_i)|^2). \quad (4.9)$$

Each term enforces the corresponding physics, initial, or boundary condition.

4.5 Network Architecture and Training

The neural network is designed as follows:

- **Input layer:** 2 neurons (x, t)
- **Hidden layers:** 5 fully connected layers with 64 neurons each
- **Activation function:** tanh
- **Output layer:** 1 neuron (u_{θ})

Training is performed using the Adam optimizer [6] with a learning rate of $\eta = 10^{-3}$ for 6,000 epochs. The collocation points, initial condition, and

boundary samples are selected as:

$$N_f = 10,000, \quad N_{IC} = 200, \quad N_{BC} = 200.$$



Figure 4.1: Training loss history over 6,000 epochs showing total, PDE, boundary, and initial losses. The Adam optimizer achieves stable convergence[7].

4.6 Results

The PINN model accurately captured the spatio-temporal dynamics of the wave equation. The predicted displacement field $u_\theta(x, t)$ showed excellent agreement with the analytical solution $u(x, t) = \sin(\pi x) \cos(\pi t)$.

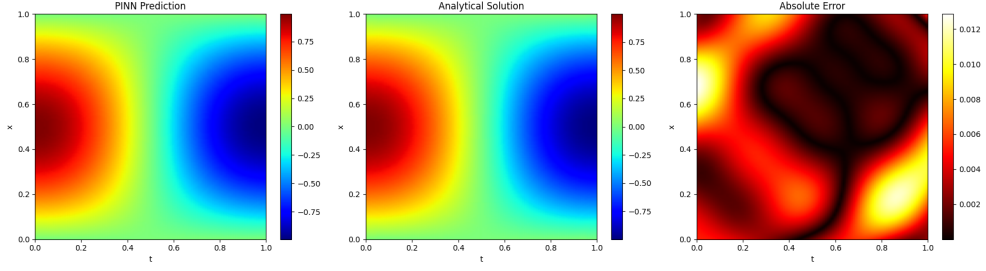


Figure 4.2: Comparison between (a) PINN-predicted solution $u_\theta(x, t)$, (b) analytical solution $u(x, t)$, and (c) absolute error $|u - u_\theta|$. The PINN accurately reproduces the full wave pattern[7].

4.6.1 Error Analysis

The relative L_2 error between predicted and analytical solutions is defined as:

$$\varepsilon_{L_2} = \frac{\|u_\theta(x, t) - u_{\text{exact}}(x, t)\|_2}{\|u_{\text{exact}}(x, t)\|_2}. \quad (4.10)$$

The trained network achieved $\varepsilon_{L_2} \approx 9.21 \times 10^{-3}$, confirming high predictive accuracy.

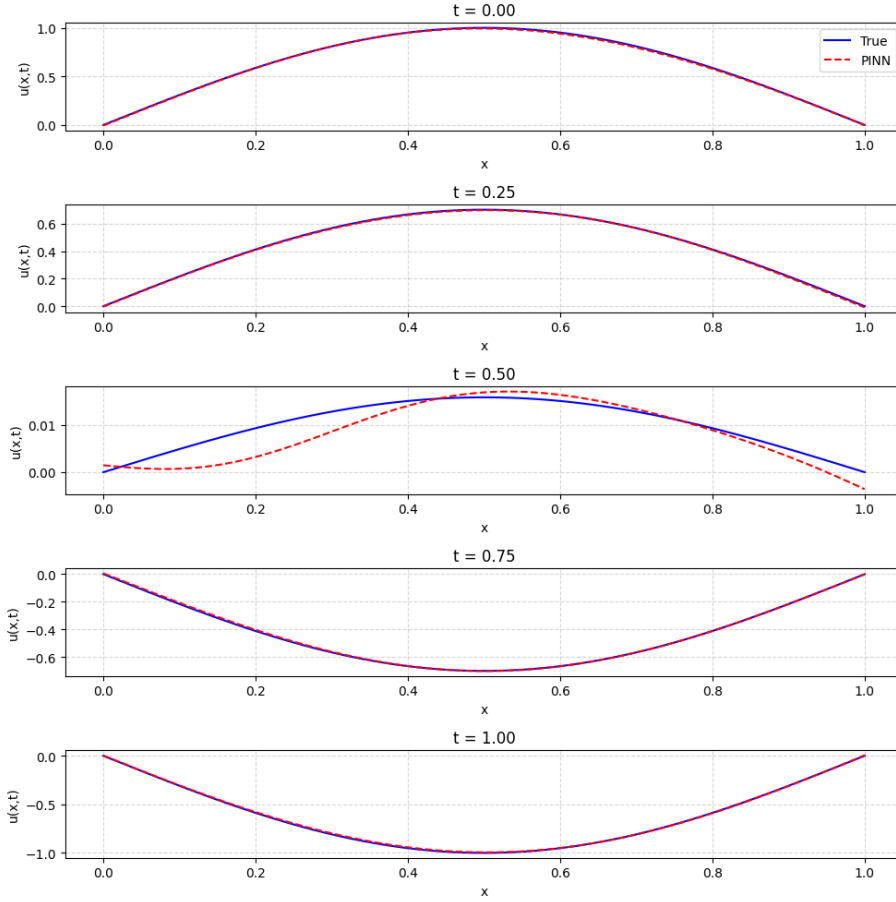


Figure 4.3: Comparison of $u(x, t)$ at different time instances ($t = 0, 0.25, 0.5, 0.75$, and 1.0). The PINN-predicted and analytical curves nearly overlap, demonstrating precise temporal predictions[7].

4.6.2 Observations

- The Adam optimizer ensured smooth and stable convergence over 6,000 epochs.
- The tanh activation effectively captured the sinusoidal nature of the wave.

- The physics-informed residual maintained consistency with the wave equation $u_{tt} = c^2 u_{xx}$.
- The model achieved a low relative error ($\varepsilon_{L_2} \approx 9.21 \times 10^{-3}$).

4.7 Conclusion

This chapter demonstrated the implementation of a Physics-Informed Neural Network for solving the one-dimensional wave equation. The model, trained with the Adam optimizer, successfully captured wave dynamics and achieved close agreement with the analytical solution. These results validate the capability of PINNs to solve linear hyperbolic PDEs in a mesh-free and data-efficient manner [11, 5, 6, 2].

Chapter 5

Conclusion and Future Direction

In Phase-I of this project, we explored the feasibility of using **deep learning methods**, particularly **Physics-Informed Neural Networks (PINNs)**, to solve physical problems governed by partial differential equations. We introduced key deep learning concepts and successfully implemented the PINN framework on classical equations such as the **one-dimensional wave equation**, demonstrating accurate and physically consistent results.

In the next phase, we plan to extend this framework to more complex systems.

Bibliography

- [1] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli. Scientific machine learning through physics-informed neural networks: Where we are and what’s next. *Journal of Scientific Computing*, 92(3):88, 2022.
- [3] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
- [4] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.
- [5] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang. Physics-informed machine learning: Theory and applications. *Nature Reviews Physics*, 3(6):422–440, 2021.
- [6] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [7] R. Kumar. Deep learning framework for solving PDEs — github repository. GitHub Repository, 2025.

- [8] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [9] M. Mohri, A. Rostamizadeh, and A. Talwalkar. *Foundations of Machine Learning*. MIT Press, 2018.
- [10] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. pages 807–814, 2010.
- [11] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [12] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [13] J. Sirignano and K. Spiliopoulos. Dgm: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018.
- [14] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.